

Synology Photo Archive Merge and Deduplication

Full technical report with reconstructed scripts, directory assumptions, dry-run methodology, and the key design lessons learned while merging a high-duplicate photo archive on Synology NAS.

Why this report exists

The practical problem was not simply duplicate removal. It was the controlled merge of two large archives where one source - staged under Backup-Photos - was heavily duplicative but still contained material worth preserving. The process therefore had to identify exact duplicates, identify files unique to the incoming set, distinguish likely format conversions such as HEIC to JPG, and then normalize the primary archive without deleting data.

This report documents the actual operational patterns used. It should be read as an engineering case study and a baseline for adaptation, not as a generic shrink-wrapped product.

Mandatory execution pattern

Every script that could modify files was intended to follow the same control flow: first produce a report or dry-run output, then review the output, then validate a sample manually, and only then allow the move. This pattern belongs at the front of the document because it explains why the scripts were written the way they were. Many of the functions that look conservative or repetitive are conservative on purpose.

Control	Implementation pattern	Why it mattered
Dry-run flag	DRY_RUN = True for move scripts	Allowed path, naming, and destination logic to be validated before modification.
CSV outputs	Report-only stage before moves	Made decisions inspectable in Excel and easy to audit.
Move-only bias	No direct deletes in operational scripts	Preserved rollback capability and reduced merge anxiety.
Directory segregation	Primary, quarantine, backup, duplicates each had dedicated locations	Reduced merge complexity and prevented mixed-state confusion.

Directory structure assumptions

The most important environmental assumption is that **/volume2/photo** is the primary archive. This report uses the term primary to mean the canonical working library - the destination intended to survive after the merge. The incoming archive was staged inside the photo share as **Backup-Photos** so that it could be compared against the canonical tree while still remaining visibly separate. That choice had consequences. Every classifier that built an index of primary content eventually had to explicitly exclude Backup-Photos or else the backup tree would be counted as proof that a file existed in the canonical archive.

The quarantine directory **Photos-Not-Found-In-Primary** was the other major working assumption. It represented material tentatively considered interesting enough to isolate for further review. Once that directory existed, it became a second stateful zone that also needed to be excluded from primary indexing. The workflow therefore relied not only on filenames and metadata but on directory semantics.

Rules of thumb

- Never delete photos during exploratory cleanup. Move them to a backup or duplicates directory instead.
- When in doubt, create a CSV. Classification is safer than intuition.
- Skip Synology index, thumbnail, and recycle content unless the task explicitly concerns those folders.

- Treat sidecars as a separate policy decision. Excluding them from matching logic may be correct, but do not forget them.
- Do not trust basename-only logic as a final answer. It is useful for discovery, not sufficient for truth.
- Do not collapse primary and backup trees into one search space when the task is to evaluate differences between them.

Metrics and observed outcomes from the merge

- The incoming source was estimated to be roughly 80% duplicate relative to the primary archive.
- One quarantine pass still revealed roughly 2,500 duplicate groups within the review set itself.
- A later exact-match cleanup in the normalized primary archive moved approximately 3,200 exact duplicates.
- A classification pass suggested that roughly half of the remaining quarantine set had no match in the primary archive once Backup-Photos was excluded.

Why the scripts evolved

The scripts did not emerge as a single plan written on paper and then executed unchanged. They evolved because each phase of the merge revealed a new class of ambiguity. The transition from simple duplicate reporting to quarantine classification happened when it became clear that the merge candidate could not be reduced to exact duplicates. The transition from quarantine classification to quarantine dedupe happened when the review set itself proved noisy. The transition to a final exact-match pass in the primary archive happened only after the noisier backup material had been moved away.

Script 1 - Duplicate report for the starting archive

Purpose. Generate a CSV listing duplicate groups based on filename, file size, and modified timestamp.

Critical learning that drove the design. The workflow needed a baseline inventory before any moves occurred. The report created a review sheet and established the report-first pattern used everywhere else.

Full script.

```
\
#!/usr/bin/env python3
import os
import csv
from pathlib import Path
from datetime import datetime

# ===== CONFIGURE =====
PRIMARY_DIR = Path("/volume2/photos")
OUTPUT_CSV = Path("/volume2/reports/duplicate_report.csv")
# =====

def format_time(ts):
    return datetime.fromtimestamp(ts).strftime('%Y-%m-%d %H:%M:%S')

def main():
    seen = {}
    duplicates = {}
    group_id = 0

    print("Scanning files...")

    for root, _, files in os.walk(PRIMARY_DIR):
        for name in files:
            path = Path(root) / name

            try:
                stat = path.stat()
            except Exception as e:
                print(f"SKIP: {path} -> {e}")
                continue

            key = (name, stat.st_size, int(stat.st_mtime))

            if key not in seen:
                seen[key] = path
            else:
                if key not in duplicates:
                    group_id += 1
                    duplicates[key] = {
                        "group_id": group_id,
                        "files": [seen[key]]
                    }
                duplicates[key]["files"].append(path)

    print(f"Writing CSV to {OUTPUT_CSV}...")

    with open(OUTPUT_CSV, "w", newline="") as csvfile:
        writer = csv.writer(csvfile)
        writer.writerow([
            "group_id",
            "type",
            "file_name",
            "full_path",
            "size_bytes",
            "modified_time"
        ])

    for entry in duplicates.values():
        files = entry["files"]
        gid = entry["group_id"]

        for i, f in enumerate(files):
            try:
                stat = f.stat()
                writer.writerow([
                    gid,
                    "original" if i == 0 else "duplicate",
                    f.name,
                    str(f),
                    stat.st_size,
                    format_time(stat.st_mtime)
                ])
            except Exception as e:
```

```
        print(f"SKIP writing: {f} -> {e}")

    print("Done.")
    print(f"Duplicate groups found: {len(duplicates)}")

if __name__ == "__main__":
    main()
```

Script 2 - Quarantine classification versus primary

Purpose. Classify each file in Photos-Not-Found-In-Primary as an exact name match, a same-basename match, or no match in the primary archive.

Critical learning that drove the design. The key insight was that 'not found' can be misleading if backup content leaks into the primary index or if basename matching is treated as exact truth.

Full script.

```
\
#!/usr/bin/env python3
import os
import csv
from pathlib import Path
from datetime import datetime

# ===== CONFIGURE =====
PHOTO_ROOT = Path("/volume2/photo")
QUARANTINE_DIR = PHOTO_ROOT / "Photos-Not-Found-In-Primary"
OUTPUT_CSV = Path("/volume2/reports/quarantine-photo-classification.csv")
# =====

IMAGE_EXTENSIONS = {
    ".jpg", ".jpeg", ".png", ".tif", ".tiff", ".bmp",
    ".gif", ".webp", ".heic", ".heif", ".dng",
    ".raw", ".cr2", ".cr3", ".nef", ".arw", ".orf", ".rw2",
    ".srw", ".raf", ".pef", ".3fr", ".erf", ".kdc", ".mrw",
    ".nrw", ".x3f"
}

SKIP_EXTENSIONS = {
    ".xmp", ".aae", ".pp3", ".dop", ".on1",
    ".txt", ".json", ".xml",
    ".db", ".ini", ".dat",
    ".thm", ".thumb",
    ".mov", ".mp4", ".avi", ".mkv", ".mts", ".m2ts",
    ".lrcat", ".lrcdata"
}

EXCLUDED_DIRS = {
    "Backup-Photos",
    "Photos-Not-Found-In-Primary"
}

SKIP_DIR_NAMES = {
    "@eaDir",
    "#recycle",
    "@tmp",
    "@__thumb",
    ".Trash",
    ".Trashes"
}

SKIP_FILE_NAMES = {
    ".DS_Store",
    "Thumbs.db",
    "desktop.ini"
}

def format_time(ts):
    return datetime.fromtimestamp(ts).strftime("%Y-%m-%d %H:%M:%S")

def is_valid_photo_file(path: Path) -> bool:
    name = path.name
    suffix = path.suffix.lower()

    if name in SKIP_FILE_NAMES or name.startswith("."):
        return False
    if suffix in SKIP_EXTENSIONS:
        return False
    if suffix not in IMAGE_EXTENSIONS:
        return False
    return True

def should_skip_dir(dir_name: str) -> bool:
    return dir_name in SKIP_DIR_NAMES or dir_name.startswith(".")

def normalized_stem(path: Path) -> str:
    return path.stem.lower()

def main():
```

```

if not PHOTO_ROOT.exists():
    raise FileNotFoundError(f"PHOTO_ROOT not found: {PHOTO_ROOT}")
if not QUARANTINE_DIR.exists():
    raise FileNotFoundError(f"QUARANTINE_DIR not found: {QUARANTINE_DIR}")

print("Indexing PRIMARY library (excluding Backup and Quarantine)...")

primary_exact = {}
primary_stem = {}

primary_count = 0
quarantine_count = 0
skipped = 0

for root, dirs, files in os.walk(PHOTO_ROOT):
    root_path = Path(root)

    pruned_dirs = []
    for d in dirs:
        if d in EXCLUDED_DIRS:
            continue
        if should_skip_dir(d):
            continue
        pruned_dirs.append(d)
    dirs[:] = pruned_dirs

    for name in files:
        path = root_path / name

        if not is_valid_photo_file(path):
            continue

        try:
            primary_exact.setdefault(path.name.lower(), []).append(path)
            primary_stem.setdefault(normalized_stem(path), []).append(path)
            primary_count += 1
        except Exception as e:
            print(f"SKIP primary: {path} -> {e}")
            skipped += 1

print("Classifying quarantine files...")

OUTPUT_CSV.parent.mkdir(parents=True, exist_ok=True)

with open(OUTPUT_CSV, "w", newline="", encoding="utf-8") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow([
        "file_name",
        "full_path",
        "relative_path",
        "extension",
        "size_bytes",
        "modified_time",
        "classification",
        "exact_match_count",
        "stem_match_count",
        "exact_match_paths",
        "stem_match_paths"
    ])

    for root, dirs, files in os.walk(QUARANTINE_DIR):
        root_path = Path(root)
        dirs[:] = [d for d in dirs if not should_skip_dir(d)]

        for name in files:
            path = root_path / name

            if not is_valid_photo_file(path):
                continue

            try:
                stat = path.stat()
                quarantine_count += 1

                exact_matches = primary_exact.get(path.name.lower(), [])
                stem_matches = primary_stem.get(normalized_stem(path), [])

                if exact_matches:
                    classification = "EXACT_NAME_MATCH_IN_PRIMARY"
                elif stem_matches:
                    classification = "SAME_BASENAME_MATCH_IN_PRIMARY"
                else:
                    classification = "NO_MATCH_IN_PRIMARY"

                writer.writerow([
                    path.name,
                    str(path),

```

```

        str(path.relative_to(QUARANTINE_DIR)),
        path.suffix.lower(),
        stat.st_size,
        format_time(stat.st_mtime),
        classification,
        len(exact_matches),
        len(stem_matches),
        " | ".join(str(p) for p in exact_matches),
        " | ".join(str(p) for p in stem_matches)
    ])

except Exception as e:
    print(f"SKIP quarantine: {path} -> {e}")
    skipped += 1

print("\nDone.")
print(f"Primary files indexed: {primary_count}")
print(f"Quarantine files classified: {quarantine_count}")
print(f"Skipped: {skipped}")
print(f"CSV written to: {OUTPUT_CSV}")

if __name__ == "__main__":
    main()

```

Script 3 - Dedupe within Photos-Not-Found-In-Primary

Purpose. Find duplicate clusters within quarantine and recommend which file to keep.

Critical learning that drove the design. The quarantine area was not inherently clean. It contained its own duplicates, so keep logic had to be defined explicitly and reviewed rather than assumed.

Full script.

```
\
#!/usr/bin/env python3
import os
import csv
from pathlib import Path
from datetime import datetime

# ===== CONFIGURE =====
TARGET_DIR = Path("/volume2/photo/Photos-Not-Found-In-Primary")
OUTPUT_CSV = Path("/volume2/reports/photos-not-found-in-primary-dedupe.csv")
# =====

IMAGE_EXTENSIONS = {
    ".jpg", ".jpeg", ".png", ".tif", ".tiff", ".bmp",
    ".gif", ".webp", ".heic", ".heif", ".dng",
    ".raw", ".cr2", ".cr3", ".nef", ".arw", ".orf", ".rw2",
    ".srw", ".raf", ".pef", ".3fr", ".erf", ".kdc", ".mrw",
    ".nrw", ".x3f"
}

SKIP_EXTENSIONS = {
    ".xmp", ".aee", ".pp3", ".dop", ".on1",
    ".txt", ".json", ".xml",
    ".db", ".ini", ".dat",
    ".thm", ".thumb",
    ".mov", ".mp4", ".avi", ".mkv", ".mts", ".m2ts",
    ".lrcat", ".lrcdata"
}

SKIP_DIR_NAMES = {
    "@eaDir",
    "#recycle",
    "@tmp",
    "@__thumb",
    ".Trash",
    ".Trashes",
    "Backup-Photos"
}

SKIP_FILE_NAMES = {
    ".DS_Store",
    "Thumbs.db",
    "desktop.ini"
}

EXTENSION_PRIORITY = {
    ".heic": 1,
    ".heif": 1,
    ".dng": 2,
    ".raw": 2,
    ".cr3": 2,
    ".cr2": 2,
    ".nef": 2,
    ".arw": 2,
    ".orf": 2,
    ".rw2": 2,
    ".raf": 2,
    ".jpg": 3,
    ".jpeg": 3,
    ".tif": 4,
    ".tiff": 4,
    ".png": 5,
    ".webp": 6,
    ".bmp": 7,
    ".gif": 8
}

def format_time(ts):
    return datetime.fromtimestamp(ts).strftime("%Y-%m-%d %H:%M:%S")

def is_valid_photo_file(path: Path) -> bool:
    name = path.name
    suffix = path.suffix.lower()
```



```

    if name in SKIP_FILE_NAMES:
        return False
    if name.startswith("."):
        return False
    if suffix in SKIP_EXTENSIONS:
        return False
    if suffix not in IMAGE_EXTENSIONS:
        return False
    return True

def should_skip_dir(dir_name: str) -> bool:
    return dir_name in SKIP_DIR_NAMES or dir_name.startswith(".")

def normalized_stem(path: Path) -> str:
    return path.stem.lower()

def extension_rank(path: Path) -> int:
    return EXTENSION_PRIORITY.get(path.suffix.lower(), 99)

def build_keep_sort_key(entry: dict):
    return (
        extension_rank(entry["path"]),
        -entry["size_bytes"],
        -entry["mtime"],
        str(entry["path"]).lower(),
    )

def recommendation(entry: dict, keep_entry: dict) -> str:
    if entry["path"] == keep_entry["path"]:
        return "KEEP"

    e_ext = entry["path"].suffix.lower()
    k_ext = keep_entry["path"].suffix.lower()

    if e_ext in {".jpg", ".jpeg"} and k_ext in {".heic", ".heif"}:
        return "REMOVE_JPG_HAS_HEIC"
    if e_ext == k_ext and entry["size_bytes"] < keep_entry["size_bytes"]:
        return "REMOVE_SMALLER_SAME_FORMAT"
    if e_ext == k_ext:
        return "REVIEW_SAME_FORMAT"
    return "REVIEW_CROSS_FORMAT"

def main():
    if not TARGET_DIR.exists():
        raise FileNotFoundError(f"TARGET_DIR not found: {TARGET_DIR}")

    print(f"Scanning only: {TARGET_DIR}")
    groups = {}
    scanned = 0
    skipped = 0

    for root, dirs, files in os.walk(TARGET_DIR):
        root_path = Path(root)
        dirs[:] = [d for d in dirs if not should_skip_dir(d)]

        for name in files:
            path = root_path / name

            if not is_valid_photo_file(path):
                continue

            try:
                stat = path.stat()
                key = normalized_stem(path)

                entry = {
                    "path": path,
                    "size_bytes": stat.st_size,
                    "mtime": stat.st_mtime,
                }

                groups.setdefault(key, []).append(entry)
                scanned += 1

            except Exception as e:
                print(f"SKIP: {path} -> {e}")
                skipped += 1

    duplicate_groups = {k: v for k, v in groups.items() if len(v) > 1}

    print("Writing CSV report...")
    OUTPUT_CSV.parent.mkdir(parents=True, exist_ok=True)

    with open(OUTPUT_CSV, "w", newline="", encoding="utf-8") as csvfile:
        writer = csv.writer(csvfile)
        writer.writerow([
            "group_id",

```

```

        "status",
        "recommendation",
        "file_name",
        "full_path",
        "relative_path",
        "extension",
        "size_bytes",
        "modified_time",
        "basename_key",
        "kept_file_path",
        "kept_extension",
        "kept_size_bytes",
        "group_file_count"
    ])

    group_id = 0

    for basename_key in sorted(duplicate_groups.keys()):
        group_id += 1
        files = sorted(duplicate_groups[basename_key], key=build_keep_sort_key)
        keep_entry = files[0]

        for idx, entry in enumerate(files):
            writer.writerow([
                group_id,
                "keep" if idx == 0 else "duplicate_candidate",
                recommendation(entry, keep_entry),
                entry["path"].name,
                str(entry["path"]),
                str(entry["path"].relative_to(TARGET_DIR)),
                entry["path"].suffix.lower(),
                entry["size_bytes"],
                format_time(entry["mtime"]),
                basename_key,
                str(keep_entry["path"]),
                keep_entry["path"].suffix.lower(),
                keep_entry["size_bytes"],
                len(files)
            ])

    print("Done.")
    print(f"Files scanned in target folder: {scanned}")
    print(f"Duplicate groups found: {len(duplicate_groups)}")
    print(f"Skipped: {skipped}")
    print(f"CSV written to: {OUTPUT_CSV}")

if __name__ == "__main__":
    main()

```

Script 4 - Move duplicate candidates back to Backup-Photos

Purpose. Use the dedupe CSV as the source of truth and move duplicate candidates back into the backup archive.

Critical learning that drove the design. CSV-driven moves turned a complicated review problem into an auditable, reversible action. It also enforced the never-delete principle.

Full script.

```
\
#!/usr/bin/env python3
import csv
import shutil
from pathlib import Path

# ===== CONFIGURE =====
CSV_PATH = Path("/volume2/reports/photos-not-found-in-primary-dedupe.csv")
SOURCE_ROOT = Path("/volume2/photo/Photos-Not-Found-In-Primary")
DEST_ROOT = Path("/volume2/photo/Backup-Photos")
DRY_RUN = True # CHANGE TO False TO ACTUALLY MOVE FILES
# =====

def safe_destination(dest_path: Path) -> Path:
    """
    Prevent overwrite by adding suffix if needed
    """
    if not dest_path.exists():
        return dest_path

    stem = dest_path.stem
    suffix = dest_path.suffix
    parent = dest_path.parent

    counter = 1
    while True:
        new_path = parent / f"{stem}__dup{counter}{suffix}"
        if not new_path.exists():
            return new_path
        counter += 1

def main():
    if not CSV_PATH.exists():
        raise FileNotFoundError(f"CSV not found: {CSV_PATH}")

    moved = 0
    skipped = 0

    print(f"Reading CSV: {CSV_PATH}")

    with open(CSV_PATH, newline="", encoding="utf-8") as csvfile:
        reader = csv.DictReader(csvfile)

        for row in reader:
            status = row.get("status")

            if status != "duplicate_candidate":
                continue

            source_path = Path(row["full_path"])

            if not source_path.exists():
                print(f"SKIP missing: {source_path}")
                skipped += 1
                continue

            try:
                rel_path = source_path.relative_to(SOURCE_ROOT)
            except Exception:
                print(f"SKIP path not under source root: {source_path}")
                skipped += 1
                continue

            dest_path = DEST_ROOT / rel_path
            dest_path.parent.mkdir(parents=True, exist_ok=True)

            dest_path = safe_destination(dest_path)

            if DRY_RUN:
                print(f"DRY RUN MOVE: {source_path} -> {dest_path}")
            else:
                print(f"MOVE: {source_path} -> {dest_path}")
                shutil.move(str(source_path), str(dest_path))
```

```
        moved += 1

    print("\nDone.")
    print(f"Files processed: {moved}")
    print(f"Skipped: {skipped}")
    print(f"DRY_RUN: {DRY_RUN}")

if __name__ == "__main__":
    main()
```

Script 5 - Exact-match cleanup in the normalized primary archive

Purpose. Remove strict exact duplicates from the canonical archive after backup noise had been isolated.

Critical learning that drove the design. The stricter the match criteria, the safer the move. Once the archive had been normalized, exact-match cleanup became the obvious high-confidence next step.

Full script.

```
\
#!/usr/bin/env python3
import os
import shutil
from pathlib import Path

# ===== CONFIGURE =====
PHOTO_ROOT = Path("/volume2/photo")
DUPLICATE_DIR = PHOTO_ROOT / "exact-matches-Duplicates"
DRY_RUN = True # Set to False to actually move files
# =====

IMAGE_EXTENSIONS = {
    ".jpg", ".jpeg", ".png", ".tif", ".tiff", ".bmp",
    ".gif", ".webp", ".heic", ".heif", ".dng",
    ".raw", ".cr2", ".cr3", ".nef", ".arw", ".orf", ".rw2",
    ".srw", ".raf", ".pef", ".3fr", ".erf", ".kdc", ".mrw",
    ".nrw", ".x3f"
}

SKIP_EXTENSIONS = {
    ".xmp", ".aae", ".pp3", ".dop", ".on1",
    ".txt", ".json", ".xml",
    ".db", ".ini", ".dat",
    ".thm", ".thumb",
    ".mov", ".mp4", ".avi", ".mkv", ".mts", ".m2ts",
    ".lrcat", ".lrcdata"
}

SKIP_DIR_NAMES = {
    "@eaDir",
    "#recycle",
    "@tmp",
    "@__thumb",
    ".Trash",
    ".Trashes",
    "exact-matches-Duplicates"
}

SKIP_FILE_NAMES = {
    ".DS_Store",
    "Thumbs.db",
    "desktop.ini"
}

def is_valid_photo_file(path: Path) -> bool:
    name = path.name
    suffix = path.suffix.lower()

    if name in SKIP_FILE_NAMES:
        return False
    if name.startswith("."):
        return False
    if suffix in SKIP_EXTENSIONS:
        return False
    if suffix not in IMAGE_EXTENSIONS:
        return False
    return True

def should_skip_dir(dir_name: str) -> bool:
    return dir_name in SKIP_DIR_NAMES or dir_name.startswith(".")

def safe_destination(src_path: Path, dest_root: Path) -> Path:
    rel = src_path.relative_to(PHOTO_ROOT)
    dest = dest_root / rel
    dest.parent.mkdir(parents=True, exist_ok=True)

    if not dest.exists():
        return dest

    stem = dest.stem
    suffix = dest.suffix
    counter = 1
    while True:
```

```

        candidate = dest.parent / f"{stem}__dup{counter}{suffix}"
        if not candidate.exists():
            return candidate
        counter += 1

def main():
    if not PHOTO_ROOT.exists():
        raise FileNotFoundError(f"PHOTO_ROOT not found: {PHOTO_ROOT}")

    print(f"Scanning for exact matches in: {PHOTO_ROOT}")
    seen = {}
    moved = 0
    skipped = 0
    scanned = 0

    for root, dirs, files in os.walk(PHOTO_ROOT):
        root_path = Path(root)
        dirs[:] = [d for d in dirs if not should_skip_dir(d)]

        for name in files:
            path = root_path / name

            if not is_valid_photo_file(path):
                continue

            try:
                stat = path.stat()
                key = (path.name.lower(), stat.st_size, int(stat.st_mtime))
                scanned += 1

                if key not in seen:
                    seen[key] = path
                    continue

                dest = safe_destination(path, DUPLICATE_DIR)

                if DRY_RUN:
                    print(f"DRY RUN MOVE: {path} -> {dest}")
                else:
                    print(f"MOVE: {path} -> {dest}")
                    shutil.move(str(path), str(dest))

                moved += 1

            except Exception as e:
                print(f"SKIP: {path} -> {e}")
                skipped += 1

    print("\nDone.")
    print(f"Files scanned: {scanned}")
    print(f"Duplicate files moved: {moved}")
    print(f"Skipped: {skipped}")
    print(f"DRY_RUN: {DRY_RUN}")
    print(f"Duplicate destination: {DUPLICATE_DIR}")

if __name__ == "__main__":
    main()

```

Appendix helper - Categorize quarantine photos into directories

Purpose. Split quarantine content into No-Match, Same-BaseName-Match, and Exact-Name-Match directories based on the classification CSV.

Critical learning that drove the design. Categorization made the review set human-readable. It translated abstract CSV labels into concrete directory structures that could be browsed in File Station.

Full script.

```
\
#!/usr/bin/env python3
import csv
import shutil
from pathlib import Path

# ===== CONFIGURE =====
CSV_PATH = Path("/volume2/reports/quarantine-photo-classification.csv")
SOURCE_ROOT = Path("/volume2/photo/Photos-Not-Found-In-Primary")
DEST_ROOT = Path("/volume2/photo")

DRY_RUN = True
# =====

CLASSIFICATION_TO_FOLDER = {
    "NO_MATCH_IN_PRIMARY": "No-Match-In-Primary",
    "SAME_BASENAME_MATCH_IN_PRIMARY": "Same-BaseName-Match-In-Primary",
    "EXACT_NAME_MATCH_IN_PRIMARY": "Exact-Name-Match-In-Primary",
}

def safe_destination(dest_path: Path) -> Path:
    if not dest_path.exists():
        return dest_path

    stem = dest_path.stem
    suffix = dest_path.suffix
    parent = dest_path.parent

    counter = 1
    while True:
        candidate = parent / f"{stem}__cat{counter}{suffix}"
        if not candidate.exists():
            return candidate
        counter += 1

def main():
    if not CSV_PATH.exists():
        raise FileNotFoundError(f"CSV not found: {CSV_PATH}")
    if not SOURCE_ROOT.exists():
        raise FileNotFoundError(f"SOURCE_ROOT not found: {SOURCE_ROOT}")
    if not DEST_ROOT.exists():
        raise FileNotFoundError(f"DEST_ROOT not found: {DEST_ROOT}")

    moved = 0
    skipped = 0
    by_class = {}

    print(f"Reading classification CSV: {CSV_PATH}")

    with open(CSV_PATH, newline="", encoding="utf-8") as csvfile:
        reader = csv.DictReader(csvfile)

        for row in reader:
            classification = row.get("classification", "").strip()
            source_path_str = row.get("full_path", "").strip()

            if classification not in CLASSIFICATION_TO_FOLDER:
                print(f"SKIP unknown classification: {classification}")
                skipped += 1
                continue

            if not source_path_str:
                print(f"SKIP missing full_path in CSV row")
                skipped += 1
                continue

            source_path = Path(source_path_str)

            if not source_path.exists():
                print(f"SKIP source missing: {source_path}")
                skipped += 1
                continue
```

```

try:
    rel_path = source_path.relative_to(SOURCE_ROOT)
except Exception:
    print(f"SKIP path not under source root: {source_path}")
    skipped += 1
    continue

category_folder = DEST_ROOT / CLASSIFICATION_TO_FOLDER[classification]
dest_path = category_folder / rel_path
dest_path.parent.mkdir(parents=True, exist_ok=True)
dest_path = safe_destination(dest_path)

if DRY_RUN:
    print(f"DRY RUN MOVE: {source_path} -> {dest_path}")
else:
    print(f"MOVE: {source_path} -> {dest_path}")
    shutil.move(str(source_path), str(dest_path))

moved += 1
by_class[classification] = by_class.get(classification, 0) + 1

print("\nDone.")
print(f"Files processed: {moved}")
print(f"Skipped: {skipped}")
print(f"DRY_RUN: {DRY_RUN}")
print("Breakdown:")
for k in sorted(by_class):
    print(f"    {k}: {by_class[k]}")

if __name__ == "__main__":
    main()

```


Appendix helper - Move orphaned XMP sidecars after image moves

Purpose. Reconcile .xmp sidecars that were left behind after image files moved into the duplicate directory.

Critical learning that drove the design. This helper exists because the main duplicate routines intentionally ignored sidecars. That was usually correct for matching, but it created a small secondary cleanup problem.

Full script.

```
\
#!/usr/bin/env python3
import os
import shutil
from pathlib import Path

# ===== CONFIGURE =====
PHOTO_ROOT = Path("/volume2/photo")
DUP_DIR = PHOTO_ROOT / "exact-matches-Duplicates"
DRY_RUN = True
# =====

IMAGE_EXTENSIONS = {
    ".jpg", ".jpeg", ".heic", ".heif", ".png", ".tif", ".tiff",
    ".dng", ".cr2", ".cr3", ".nef", ".arw", ".orf", ".rw2"
}

def find_matching_image(base_path: Path):
    stem = base_path.stem

    for root, _, files in os.walk(DUP_DIR):
        for f in files:
            p = Path(root) / f
            if p.stem == stem and p.suffix.lower() in IMAGE_EXTENSIONS:
                return p
    return None

def main():
    moved = 0
    skipped = 0

    print("Scanning for orphaned XMP files...")

    for root, _, files in os.walk(PHOTO_ROOT):
        for name in files:
            if not name.lower().endswith(".xmp"):
                continue

            xmp_path = Path(root) / name

            matching_local = any(
                (Path(root) / (xmp_path.stem + ext)).exists()
                for ext in IMAGE_EXTENSIONS
            )

            if matching_local:
                continue

            match = find_matching_image(xmp_path)

            if not match:
                continue

            rel = match.relative_to(DUP_DIR)
            dest = DUP_DIR / rel.parent / (xmp_path.stem + ".xmp")
            dest.parent.mkdir(parents=True, exist_ok=True)

            if DRY_RUN:
                print(f"DRY RUN MOVE: {xmp_path} -> {dest}")
            else:
                print(f"MOVE: {xmp_path} -> {dest}")
                shutil.move(str(xmp_path), str(dest))

            moved += 1

    print("\nDone.")
    print(f"XMP files moved: {moved}")
    print(f"Skipped: {skipped}")
    print(f"DRY_RUN: {DRY_RUN}")

if __name__ == "__main__":
    main()
```

Command workflow examples

The scripts were typically created under a scripts directory and then invoked directly with python3. Representative commands used in the workflow are shown below.

```
nano /volume2/scripts/find_duplicates_report.py
python3 /volume2/scripts/find_duplicates_report.py

nano /volume2/scripts/quarantine_photo_classification.py
python3 /volume2/scripts/quarantine_photo_classification.py

nano /volume2/scripts/dedupe_photos_not_found_in_primary.py
python3 /volume2/scripts/dedupe_photos_not_found_in_primary.py

nano /volume2/scripts/move_duplicates_back_to_backup.py
python3 /volume2/scripts/move_duplicates_back_to_backup.py

nano /volume2/scripts/move_exact_matches_in_photo.py
python3 /volume2/scripts/move_exact_matches_in_photo.py
```

Synology-specific learnings

- Backup directories inside the photo share are operationally convenient but analytically dangerous unless they are excluded explicitly from primary indexing.
- Synology metadata directories such as @eaDir can inflate counts and mislead filesystem-oriented scripts unless excluded early and consistently.
- Recycle bin growth on a secondary NAS can become material when large delete-equivalent moves propagate through ShareSync.
- File Station visibility differs from underlying Linux paths, so a user-facing folder name may correspond to /volume2/... beneath the DSM abstraction layer.

What not to assume from these scripts

These scripts do not prove content identity by hashing. They do not infer Lightroom catalog state. They do not guarantee that a JPG and a HEIC with the same basename are semantically equivalent. They also do not attempt to preserve sidecar relationships automatically in every move path. Their value is practical and bounded: they provide a set of structured, reviewable operations for a very specific merge design.

Anyone adapting them should re-examine directory assumptions, extension lists, desired keep policy, and the interaction between image files and sidecars. The scripts are best treated as baseline implementations that accelerate thought and reduce typing, not as turnkey products.

Closing perspective

The central achievement of this workflow was not a single script. It was the conversion of a messy merge into a staged data-reconciliation process with rollback. That required explicit assumptions about /volume2/photo as the canonical library, explicit handling of Backup-Photos as an incoming set to be merged, report-first execution, and the discipline to move rather than delete. That is the real reusable pattern captured here.